Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы трансляции»

ОТЧЕТ
к лабораторной работе №2
на тему:
«ЛЕКСИЧЕСКИЙ АНАЛИЗ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Описание работы программы 4
3 Ход выполнения программы 5
Вывод 9
Список использованных источников 10
Приложение А (справочное) Исходный код 11

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Разработка лексического анализатора подмножества языка программирования, определенного в лабораторной работе 1. Определяются лексические правила. Выполняется перевод потока символов в поток лексем (токенов).

Лексический анализатор читает поток символов, составляющих исходную программу, и группирует эти символы в значащие последовательности, называемые лексемами [1]. На вход программы подается текстовый файл (напр. с именем INPUT.TXT), содержащий строки символов анализируемой программы.

После того, как в результате лексического анализа лексемы распознаны, информация о некоторых из них собирается и записывается в таблицу имен.

Если позднее во входной цепочке попадается идентификатор, надо справиться в этой таблице, не появлялся ли он ранее. Если да, то лексема, соответствующая новому вхождению этого идентификатора, будет той же, что и у предыдущего вхождения. Необходимо создание таблиц констант, ключевых слов, разделителей, логических/математических операторов. При определении неверной последовательности символов, необходимо обнаружить эту ошибку и выдать сообщение о ней. Всего необходимо показать скриншоты нахождения 4-х лексических ошибок.

2 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

В ходе выполнения лабораторной работы был разработан лексический анализатор для подмножества языка программирования Idris, предназначенного для разбора исходного кода, представленного в текстовом файле input.txt. Задача заключалась в том, чтобы прочитать исходный код, выделить лексемы и классифицировать их по типам: ключевые слова, идентификаторы, типы данных, константы, операторы, разделители, а также выполнить проверки на лексические ошибки.

Первоначально был создан набор ключевых слов и типов данных. Эти элементы использовались для определения типа каждой встреченной лексемы в процессе лексического анализа. Для обработки числовых констант был реализован детерминированный автомат [2], распознающий целые числа, числа с плавающей точкой и числа в экспоненциальной записи, такие как 3.14159, 1.2.3 и 10e, с последующим выявлением ошибок в некорректных форматах. Строковые и символьные литералы обрабатывались с проверкой на закрывающие кавычки, что позволило обнаружить незакрытые строки, как в примере error4. Для каждого типа лексемы были созданы соответствующие таблицы: идентификаторов, констант, ключевых слов, типов данных, операторов и разделителей. При обнаружении новой лексемы проверялось её наличие в соответствующей таблице; если элемент отсутствовал, он добавлялся с уникальным индексом. Это обеспечило устранение дублирования и структурированное хранение информации.

В процессе анализа также выявлялись лексические ошибки, такие как некорректные числа (например, 1.2.3 и 10e), недопустимые символы (например, #) и незакрытые строковые литералы. Для каждой ошибки фиксировалась строка и позиция в исходном коде, что позволило вывести подробные сообщения об ошибках. После завершения лексического анализа программа выводила таблицу всех найденных лексем с указанием типа, значения, строки и позиции, а также отдельные таблицы для каждого класса лексем и статистику по количеству токенов и ошибок.

3 ХОД ВЫПОЛНЕНИЯ ПРОГРАММЫ

В ходе выполнения программы лексический анализатор последовательно обрабатывает исходный код из файла input.txt. На первом этапе анализатор распознаёт ключевые слова, такие как import, let, if, then, else, case, of, do, putStrLn, show, main, insert, empty [3]. Все найденные ключевые слова заносятся в таблицу, где каждому из них присваивается уникальный числовой идентификатор. Вывод таблицы рассмотрим ниже, на рисунке 3.1.

```
Keywords:
----------------------------------------
ID   VALUE
----------------------------------------
1    import
2    let
3    in
4    if
5    then
6    else
7    case
8    of
9    putStrLn
10   do
11   show
12   main
13   insert
14   empty
```

Рисунок 3.1 – Вывод ключевых слов из файла input.txt

Далее анализатор идентифицирует типы данных, встречающиеся в коде: Vect, SortedMap, List, Int, Double, String, Nat, IO, Char, Integer, Bits8. Они также регистрируются в отдельной таблице с соответствующими порядковыми номерами, что отражено на соответствующем рисунке 3.2.

```
Data Types:
----------------------------------------
ID   VALUE
----------------------------------------
1    Vect
2    SortedMap
3    List
4    Int
5    Double
6    String
7    Nat
8    IO
9    Char
10   Integer
11   Bits8
```

Рисунок 3.2 – Вывод типов данных из файла input.txt

Затем программа выделяет все идентификаторы, включая имена модулей, функций, переменных и параметров, такие как Data, numbersList, checkValue, x, d, n и другие. Каждому идентификатору присваивается уникальный номер, и формируется таблица, представленная на рисунке 3.3.



```
Identifiers:
----------------------------------------
ID   VALUE
----------------------------------------
1    Data
2    numbersList
3    checkValue
4    x
5    d
6    n
7    cast
8    processData
9    id
10   name
11   vec
12   _
13   forLoop
14   Z
15   S
16   k
17   result
18   myPair
19   myVec
20   myChar
21   bigInt
22   bVal
23   dict
24   error1
25   error2
26   error3
27   error4
```

Рисунок 3.3 – Вывод идентификаторов из файла input.txt

После этого обрабатываются константы: целые числа (например, 10, 20, 30, 5, 0.0, 3, 1, 2, 42, 255), числа с плавающей точкой (3.14159), большие целые (12345678901234567890), строковые литералы (такие как "Greater than 5 and Double is positive", "Condition not met", "Idris", "Language match!", "Unknown", "Done", "Iteration:", "key") и символьные литералы ('A'). Все они заносятся в таблицу, что можно увидеть на соответствующем рисунке 3.4.

```
Constants:
---------------------------------------
ID   VALUE
---------------------------------------
1    10
2    20
3    30
4    5
5    0.0
6    Greater than 5 and Double is positive
7    Condition not met
8    3
9    Idris
10   Language match!
11   Unknown
12   Done
13   Iteration:
14   3.14159
15   1
16   2
17   A
18   12345678901234567890
19   255
20   key
21   42
```

Рисунок 3.4 – Вывод констант из файла input.txt

Операторы, включая :, =, ->, >, &&, =>, +, регистрируются в таблице операторов, что можно рассмотреть на рисунке 3.5.

```
Operators:
---------------------------------------
ID   VALUE
---------------------------------------
1    :
2    =
3    ->
4    >
5    &&
6    =>
7    +
```

Рисунок 3.5 – Вывод операторов из файла input.txt

А, также, разделители языка в отдельной таблице приведены на рисунке 3.6.

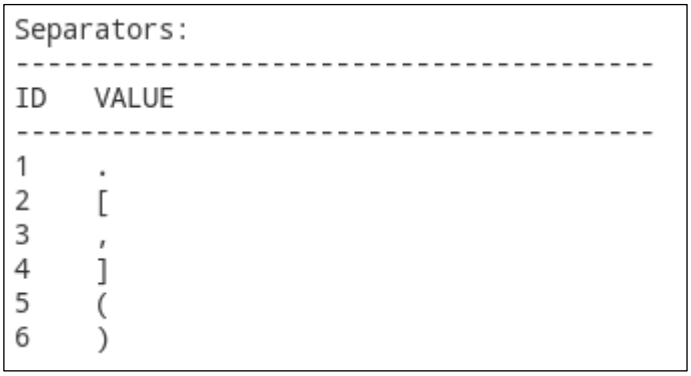


Рисунок 3.6 – Вывод разделителей из файла input.txt

Лексемы формируются в массив, который требуется вывести, как последовательность лексем, что представлено на рисунке 3.7.

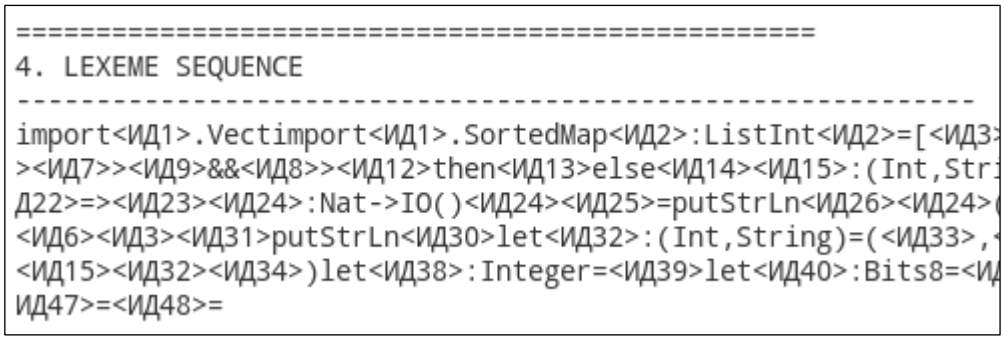


Рисунок 3.7 – Вывод последовательности лексем

Согласно условию лабораторной работы, были обработаны, также, ошибки с помощью лексического анализатора, что показано на рисунке 3.8.

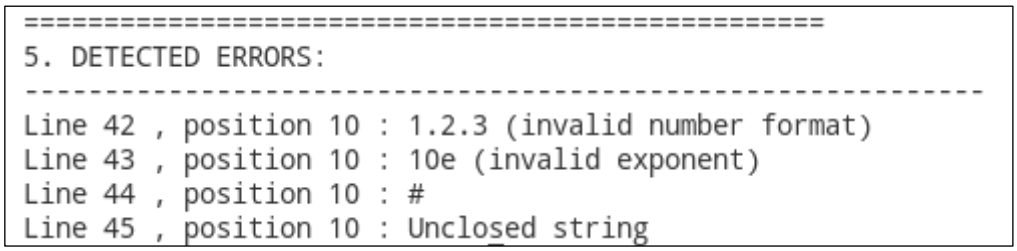


Рисунок 3.8 – Вывод лексических ошибок в коде

ВЫВОД

В ходе выполнения лабораторной работы был успешно разработан лексический анализатор для подмножества языка программирования Idris. Программа корректно выполняет чтение исходного кода из файла, выделяет лексемы и классифицирует их по типам: ключевые слова, идентификаторы, типы данных, константы, операторы и разделители. Были созданы соответствующие таблицы символов, что позволяет группировать информацию об элементах исходного кода.

Анализатор также обнаруживает лексические ошибки, такие как некорректные числовые форматы, недопустимые символы и незакрытые строковые литералы. Каждая ошибка фиксируется с указанием строки и позиции.

Реализованный лексический анализатор демонстрирует полное соответствие поставленным задачам: он корректно разбирает типичные конструкции языка Idris, заполняет таблицы идентификаторов, констант, ключевых слов и других категорий лексем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Лексический анализ в разработке компилятора с примером [Электронный ресурс]. – Режим доступа: https://www.guru99.com/ru/compiler-design-lexical-analysis.html. – Дата доступа: 08.02.2026.

[2] Детерминированные автоматы [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/358304/?ysclid=mkz981lgu3457596025. – Дата доступа: 08.02.2026.

[3] Документация по Idris [Электронный ресурс]. – Режим доступа: https://docs.idris-lang.org/en/latest. – Дата доступа: 08.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация лексического анализатора

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <cctype>
#include <sstream>
#include <iomanip>

using namespace std;

enum TokenType
{
    KEYWORD,
    IDENTIFIER,
    TYPE,
    CONSTANT,
    OPERATOR,
    SEPARATOR,
    STRING_LITERAL,
    CHAR_LITERAL,
    ERROR
};

struct Token
{
    TokenType type;
    string value;
    int line;
    int position;
};

class IdrisLexer
{
private:
    string sourceCode;
    int position;
    int line;
    int column;
    vector<Token> tokens;

    vector<string> keywords = {
        "let", "in", "do", "if", "then", "else", "case", "of",
        "import", "main", "putStrLn", "show", "insert", "empty"};

    vector<string> types = {
        "Int", "Double", "String", "List", "Vect", "Nat", "IO",
        "Char", "Integer", "Bits8", "SortedMap"};

    char peek()
    {
        if (position >= sourceCode.length())
            return '\0';
        return sourceCode[position];
    }
```

```
    char advance()
    {
        if (position >= sourceCode.length())
            return '\0';
        char c = sourceCode[position];
        position++;
        column++;
        return c;
    }

    void addToken(TokenType type, string value)
    {
        Token token;
        token.type = type;
        token.value = value;
        token.line = line;
        token.position = column - value.length();
        tokens.push_back(token);
    }

    bool isKeyword(string word)
    {
        for (const auto &kw : keywords)
        {
            if (kw == word)
                return true;
        }
        return false;
    }

    bool isType(string word)
    {
        for (const auto &t : types)
        {
            if (t == word)
                return true;
        }
        return false;
    }

    void skipWhitespace()
    {
        while (true)
        {
            char c = peek();
            if (c == ' ' || c == '\t' || c == '\r' || c == '(' || c == ')' ||
c == '[' || c == ']' || c == '{' || c == '}')
            {
                advance();
            }
            else if (c == '\n')
            {
                advance();
                line++;
                column = 1;
            }
            else
            {
                break;
            }
        }
    }
```

```
    void processIdentifier()
    {
        int start = position;
        if (isalpha(peek()) || peek() == '_')
        {
            advance();
        } else {
            addToken(ERROR, string(1, advance()));
            return;
        }

        while (isalnum(peek()) || peek() == '_')
        {
            advance();
        }

        string ident = sourceCode.substr(start, position - start);

        if (isKeyword(ident))
        {
            addToken(KEYWORD, ident);
        }
        else if (isType(ident))
        {
            addToken(TYPE, ident);
        }
        else
        {
            addToken(IDENTIFIER, ident);
        }
    }

    void processNumber()
    {
        int start = position;
        string numStr;

        enum class State { START, INTEGER, DOT, FRACTION, EXP, SIGN, EXP_DIGITS
};
        State state = State::START;

        while (position < sourceCode.length()) {
            char c = peek();
            switch (state) {
                case State::START:
                    if (isdigit(c)) {
                        state = State::INTEGER;
                        numStr += advance();
                    } else {
                        goto end_loop;
                    }
                    break;
                case State::INTEGER:
                    if (isdigit(c)) {
                        numStr += advance();
                    } else if (c == '.') {
                        state = State::DOT;
                        numStr += advance();
                    } else if (c == 'e' || c == 'E') {
                        state = State::EXP;
                        numStr += advance();
                    } else {
                        goto end_loop;
                    }
```

```
                    break;
                case State::DOT:
                    if (isdigit(c)) {
                        state = State::FRACTION;
                        numStr += advance();
                    } else {
                        addToken(ERROR, numStr + " (incomplete fraction)");
                        return;
                    }
                    break;
                case State::FRACTION:
                    if (isdigit(c)) {
                        numStr += advance();
                    } else if (c == 'e' || c == 'E') {
                        state = State::EXP;
                        numStr += advance();
                    } else if (c == '.') { // Second dot
                        numStr += advance();
                        while(isdigit(peek())){
                            numStr += advance();
                        }
                        addToken(ERROR, numStr + " (invalid number format)");
                        return;
                    }
                    else {
                        goto end_loop;
                    }
                    break;
                case State::EXP:
                    if (c == '+' || c == '-') {
                        state = State::SIGN;
                        numStr += advance();
                    } else if (isdigit(c)) {
                        state = State::EXP_DIGITS;
                        numStr += advance();
                    } else {
                        addToken(ERROR, numStr + " (invalid exponent)");
                        return;
                    }
                    break;
                case State::SIGN:
                    if (isdigit(c)) {
                        state = State::EXP_DIGITS;
                        numStr += advance();
                    } else {
                        addToken(ERROR, numStr + " (exponent missing digits)");
                        return;
                    }
                    break;
                case State::EXP_DIGITS:
                    if (isdigit(c)) {
                        numStr += advance();
                    } else {
                        goto end_loop;
                    }
                    break;
            }
        }

    end_loop:
        if (state == State::INTEGER || state == State::FRACTION || state ==
State::EXP_DIGITS) {
            addToken(CONSTANT, numStr);
        }
```

```
    else {
        addToken(ERROR, numStr + " (incomplete number)");
    }
}

void processString()
{
    advance();
    int start = position;

    while (peek() != '"' && peek() != '\0')
    {
        advance();
    }

    string str = sourceCode.substr(start, position - start);

    if (peek() == '"')
    {
        advance();
        addToken(STRING_LITERAL, str);
    }
    else
    {
        addToken(ERROR, "Unclosed string");
    }
}

void processChar()
{
    advance();
    char c = advance();

    if (peek() == '\'')
    {
        advance();
        addToken(CHAR_LITERAL, string(1, c));
    }
    else
    {
        addToken(ERROR, "Unclosed character literal");
    }
}

void processOperator()
{
    char c = advance();
    string op(1, c);

    if (c == ':' && peek() == ':')
    {
        advance();
        op = "::";
    }
    else if (c == '-' && peek() == '>')
    {
        advance();
        op = "->";
    }
    else if (c == '&' && peek() == '&')
    {
        advance();
        op = "&&";
    }
```

```
        else if (c == '=' && peek() == '>')
        {
            advance();
            op = "=>";
        }

        addToken(OPERATOR, op);
    }

public:
    IdrisLexer(const string &code) : sourceCode(code), position(0), line(1),
column(1) {}

    vector<Token> analyze()
    {
        while (position < sourceCode.length())
        {
            skipWhitespace();
            char c = peek();

            if (c == '\0')
                break;

            if (isalpha(c) || c == '_')
            {
                processIdentifier();
            }
            else if (isdigit(c))
            {
                processNumber();
            }
            else if (c == '"')
            {
                processString();
            }
            else if (c == '\'')
            {
                processChar();
            }
            else if (c == ',' || c == ';' || c == '.')
            {
                addToken(SEPARATOR, string(1, advance()));
            }
            else if (c == ':' || c == '=' || c == '>' || c == '&' || c == '+'
|| c == '*' || c == '-')
            {
                processOperator();
            }
            else
            {
                addToken(ERROR, string(1, advance()));
            }
        }

        return tokens;
    }
};
```